

Trabalho de LLP - Sophia Carvalho - EIEL21a

1. É um jeito de organizar e guardar dados na memória para que o programa consiga acessá-los, alterá-los e removê-los de maneira eficiente. Em vez de criar uma variável solta para cada informação, agrupamos os dados em uma estrutura com regras de organização.

Exemplo: um programa de notas escolares que guarda as notas de 30 alunos em um vetor (`float notas[30];`) em vez de criar 30 variáveis (`nota1`, `nota2...`).

2. Vetor é uma estrutura que guarda vários valores do mesmo tipo sob um único nome. Os elementos ficam em posições de memória consecutivas (uma colada na outra) e cada um é identificado por um índice, que começa em 0. Um vetor de 5 posições tem índices de 0 a 4. Como as posições são vizinhas, o computador calcula onde cada elemento está sem precisar procurar.

- 3. a) `valores[0] = 4`
- b) `valores[3] = 16`
- c) O número 20 está no índice 4
- d) Existem 5 elementos

4. No vetor, os elementos ficam juntos e em sequência na memória, com tamanho fixo. Na lista encadeada, os elementos (nós) ficam espalhados pela memória e cada um guarda o endereço do próximo, então o tamanho pode crescer ou diminuir durante a execução.

5. Nó é cada "elemento" da lista encadeada. Normalmente tem duas partes: o dado (valor armazenado, como um `int`) e um ponteiro para o próximo nó.

6. Ele guarda o endereço de memória do nó seguinte. É esse ponteiro que "liga" os nós. Sem ele não daria para andar pela lista, porque os nós não estão lado a lado na memória.

7. Porque o último nó não tem ninguém depois dele. O `nullptr` funciona como um sinal de "fim da lista". Quando percorremos a lista, paramos ao encontrar um ponteiro nulo. Se ele apontasse para um endereço qualquer, o programa tentaria ler memória que não pertence à lista e poderia travar.

8. Vetor: (1) acesso direto e rápido a qualquer elemento pelo índice; (2) uso de memória mais simples, pois não precisa guardar ponteiros.

Lista encadeada: (1) tamanho dinâmico, pois cresce e diminui conforme a necessidade; (2) inserir e remover é fácil, porque basta mudar ponteiros, sem deslocar os outros elementos.

9.

```
#include <iostream>
using namespace std;

int main() {
    int v[10];

    // Leitura dos 10 valores
    for (int i = 0; i < 10; i++) {
        cout << "Digite o numero " << i + 1 << ": ";
        cin >> v[i];
    }

    // Exibição
    cout << "\nValores armazenados:\n";
    for (int i = 0; i < 10; i++) {
        cout << "v[" << i << "] = " << v[i] << endl;
    }
    return 0;
}
```

10.

```
#include <iostream>
using namespace std;

int main() {
    int v[10];
    for (int i = 0; i < 10; i++) {
        cout << "Digite o numero " << i + 1 << ": ";
        cin >> v[i];
    }

    int maior = v[0], menor = v[0];
    int posMaior = 0, posMenor = 0;

    for (int i = 1; i < 10; i++) {
        if (v[i] > maior) {
            maior = v[i];
            posMaior = i;
        }
        if (v[i] < menor) {
            menor = v[i];
            posMenor = i;
        }
    }
}
```

```

    }
}

    cout << "Maior valor: " << maior << " (posicao " << posMaior <<
"\n";
    cout << "Menor valor: " << menor << " (posicao " << posMenor <<
"\n";
    return 0;
}

```

11.

```

#include <iostream>
using namespace std;

int main() {
    float notas[5];
    float soma = 0;

    for (int i = 0; i < 5; i++) {
        cout << "Nota do aluno " << i + 1 << ": ";
        cin >> notas[i];
        soma += notas[i];
    }

    float media = soma / 5;
    float maior = notas[0], menor = notas[0];
    int acima = 0;

    for (int i = 0; i < 5; i++) {
        if (notas[i] > maior) maior = notas[i];
        if (notas[i] < menor) menor = notas[i];
        if (notas[i] > media) acima++;
    }

    cout << "\nNotas: ";
    for (int i = 0; i < 5; i++) cout << notas[i] << " ";

    cout << "\nMedia da turma: " << media;
    cout << "\nMaior nota: " << maior;
    cout << "\nMenor nota: " << menor;
    cout << "\nNotas acima da media: " << acima << endl;
    return 0;
}

```

12.

```
#include <iostream>
using namespace std;

int main() {
    int v[10];
    int pares = 0, impares = 0;

    for (int i = 0; i < 10; i++) {
        cout << "Digite o numero " << i + 1 << ": ";
        cin >> v[i];
        if (v[i] % 2 == 0) pares++;
        else impares++;
    }

    cout << "Pares: " << pares << endl;
    cout << "Impares: " << impares << endl;
    return 0;
}
```

13.

```
#include <iostream>
using namespace std;

int main() {
    int v[10], busca;
    bool achou = false;

    for (int i = 0; i < 10; i++) {
        cout << "Digite o numero " << i + 1 << ": ";
        cin >> v[i];
    }

    cout << "Qual numero deseja procurar? ";
    cin >> busca;

    for (int i = 0; i < 10; i++) {
        if (v[i] == busca) {
            cout << "Valor encontrado na posicao " << i << endl;
            achou = true;
            break; // encontrou o primeiro, pode parar
        }
    }
}
```

```

    }

    if (!achou) cout << "Valor nao encontrado" << endl;
    return 0;
}

```

14.

- a) Primeiro elemento: 10
- b) Último elemento: 45
- c) Depois do 25 vem o 30
- d) Existem 4 nós
- e) O nó 45 aponta para NULL

15.

Inserindo 5 no início (o novo nó passa a ser a cabeça e aponta para o antigo primeiro):

5 → 10 → 20 → 30 → NULL

Inserindo 40 no final (o antigo último, o 30, passa a apontar para o 40, e o 40 aponta para NULL):

5 → 10 → 20 → 30 → 40 → NULL

16.

Lista inicial: 5 → 10 → 20 → 30 → 40 → NULL

- a) Remover 5 (início; a cabeça passa a ser o 10):
10 → 20 → 30 → 40 → NULL
- b) Remover 30 (o 20 passa a apontar para o 40):
10 → 20 → 40 → NULL
- c) Remover 40 (o 20 passa a apontar para NULL):
10 → 20 → NULL

17.

```

struct No {
    int valor;
    No* proximo;
};

void inserirInicio(No*& inicio, int valor) {
    No* novo = new No;      // cria o novo nó
    novo->valor = valor;     // guarda o valor

```

```

        novo->proximo = inicio; // novo aponta para o antigo primeiro
        inicio = novo;          // novo vira o primeiro
    }

```

18.

```

void listar(No* inicio) {
    No* atual = inicio;
    while (atual != nullptr) {
        cout << atual->valor << " -> ";
        atual = atual->proximo; // anda para o próximo nó
    }
    cout << "NULL" << endl;
}

```

19.

```

bool buscar(No* inicio, int valor) {
    No* atual = inicio;
    while (atual != nullptr) {
        if (atual->valor == valor) return true;
        atual = atual->proximo;
    }
    return false;
}

```

20.

```

#include <iostream>
using namespace std;

```

```

const int N = 10;

```

```

void cadastrar(float notas[]) {
    for (int i = 0; i < N; i++) {
        cout << "Nota do aluno " << i + 1 << ": ";
        cin >> notas[i];
    }
}

```

```

void mostrar(float notas[]) {
    for (int i = 0; i < N; i++) {
        cout << "Aluno " << i + 1 << ": " << notas[i] << endl;
    }
}

```

```

float calcularMedia(float notas[]) {
    float soma = 0;
    for (int i = 0; i < N; i++) soma += notas[i];
    return soma / N;
}

int main() {
    float notas[N];
    int opcao;
    bool cadastrado = false;

    do {
        cout << "\n--- SISTEMA DE NOTAS ---\n";
        cout << "1 - Cadastrar notas\n";
        cout << "2 - Mostrar notas\n";
        cout << "3 - Media da turma\n";
        cout << "4 - Maior e menor nota\n";
        cout << "5 - Alunos acima da media\n";
        cout << "6 - Procurar uma nota\n";
        cout << "0 - Sair\n";
        cout << "Opcao: ";
        cin >> opcao;

        if (opcao >= 2 && opcao <= 6 && !cadastrado) {
            cout << "Cadastre as notas primeiro (opcao 1)!\n";
            continue;
        }

        switch (opcao) {
            case 1:
                cadastrar(notas);
                cadastrado = true;
                break;
            case 2:
                mostrar(notas);
                break;
            case 3:
                cout << "Media: " << calcularMedia(notas) << endl;
                break;
            case 4: {
                float maior = notas[0], menor = notas[0];
                for (int i = 1; i < N; i++) {
                    if (notas[i] > maior) maior = notas[i];

```

```

        if (notas[i] < menor) menor = notas[i];
    }
    cout << "Maior: " << maior << " | Menor: " << menor
<< endl;
    break;
}
case 5: {
    float media = calcularMedia(notas);
    int cont = 0;
    for (int i = 0; i < N; i++)
        if (notas[i] > media) cont++;
    cout << cont << " aluno(s) acima da media (" <<
media << ")\n";
    break;
}
case 6: {
    float busca;
    bool achou = false;
    cout << "Nota procurada: ";
    cin >> busca;
    for (int i = 0; i < N; i++) {
        if (notas[i] == busca) {
            cout << "Encontrada no aluno " << i + 1 <<
endl;
            achou = true;
        }
    }
    if (!achou) cout << "Nota nao encontrada\n";
    break;
}
case 0:
    cout << "Encerrando...\n";
    break;
default:
    cout << "Opcao invalida!\n";
}
} while (opcao != 0);

return 0;
}

```

21.

```
#include <iostream>
```

```

using namespace std;

struct No {
    int valor;
    No* proximo;
};

// Insere no FINAL da lista
void inserir(No*& inicio, int valor) {
    No* novo = new No;
    novo->valor = valor;
    novo->proximo = nullptr;

    if (inicio == nullptr) {          // lista vazia
        inicio = novo;
    } else {
        No* atual = inicio;
        while (atual->proximo != nullptr) // vai até o último
            atual = atual->proximo;
        atual->proximo = novo;
    }
}

void listar(No* inicio) {
    if (inicio == nullptr) {
        cout << "Lista vazia.\n";
        return;
    }
    for (No* atual = inicio; atual != nullptr; atual =
atual->proximo)
        cout << atual->valor << " -> ";
    cout << "NULL\n";
}

bool buscar(No* inicio, int valor) {
    for (No* atual = inicio; atual != nullptr; atual =
atual->proximo)
        if (atual->valor == valor) return true;
    return false;
}

bool remover(No*& inicio, int valor) {
    No* atual = inicio;

```

```

No* anterior = nullptr;

while (atual != nullptr && atual->valor != valor) {
    anterior = atual;
    atual = atual->proximo;
}
if (atual == nullptr) return false; // não achou

if (anterior == nullptr)           // removendo o primeiro
    inicio = atual->proximo;
else
    anterior->proximo = atual->proximo;

delete atual;                       // libera a memória
return true;
}

int contar(No* inicio) {
    int total = 0;
    for (No* atual = inicio; atual != nullptr; atual =
atual->proximo)
        total++;
    return total;
}

void liberarLista(No*& inicio) {
    while (inicio != nullptr) {
        No* temp = inicio;
        inicio = inicio->proximo;
        delete temp;
    }
}

int main() {
    No* lista = nullptr;
    int opcao, valor;

    do {
        cout << "\n1 - Inserir elemento\n";
        cout << "2 - Listar elementos\n";
        cout << "3 - Buscar elemento\n";
        cout << "4 - Remover elemento\n";
        cout << "5 - Contar elementos\n";
    }

```

```

    cout << "0 - Sair\n";
    cout << "Opcao: ";
    cin >> opcao;

    switch (opcao) {
        case 1:
            cout << "Valor a inserir: ";
            cin >> valor;
            inserir(lista, valor);
            break;
        case 2:
            listar(lista);
            break;
        case 3:
            cout << "Valor a buscar: ";
            cin >> valor;
            cout << (buscar(lista, valor) ? "Encontrado!" : "Nao
encontrado.") << endl;
            break;
        case 4:
            cout << "Valor a remover: ";
            cin >> valor;
            cout << (remover(lista, valor) ? "Removido!" :
"Valor nao existe na lista.") << endl;
            break;
        case 5:
            cout << "Total de elementos: " << contar(lista) <<
endl;
            break;
        case 0:
            cout << "Saindo...\n";
            break;
        default:
            cout << "Opcao invalida!\n";
    }
    } while (opcao != 0);

    liberarLista(lista);
    return 0;
}

```

Como os registros são adicionados e removidos o tempo todo, a estrutura mais adequada é a lista encadeada.

No vetor, o tamanho é fixo. Se ele encher, é preciso criar um vetor maior e copiar tudo. Remover um registro do meio deixa um "buraco", então todos os elementos seguintes precisam ser deslocados uma posição, e isso fica lento com muitos dados. A vantagem do vetor é o acesso direto pelo índice ($v[500]$ é imediato). A desvantagem é o custo de inserir e remover no meio e o tamanho fixo.

Na lista encadeada, inserir ou remover só exige mudar alguns ponteiros, sem mexer nos outros elementos, e o tamanho se ajusta sozinho. A vantagem é essa flexibilidade. A desvantagem é que não existe acesso direto: para chegar ao 500º elemento é preciso passar pelos 499 anteriores. Além disso, cada nó gasta memória extra com o ponteiro.

Se o programa mexe mais em inserções e remoções do que em consultas por posição, a lista compensa. Se o principal fosse consultar registros por posição, o vetor seria melhor.